# Solution Architecture

Project: Rising Water – AI-Based Flood Prediction System

## 1. Architecture Overview

The Rising Water system follows a layered architecture consisting of a user interface, Flask application, preprocessing layer, trained XGBoost model, and prediction output.

## 2. System Architecture Diagram

```
+---------------------+
|     User            |
+----------+----------+
           |
           v
+---------------------+
| HTML / CSS Frontend |
+----------+----------+
           |
           v
+---------------------+
|  Flask Application  |
+----------+----------+
           |
           v
+---------------------+
| Input Validation &  |
| Feature Scaling     |
+----------+----------+
           |
           v
+---------------------+
| XGBoost ML Model    |
+----------+----------+
           |
           v
+---------------------+
| Flood Probability & |
| Risk Level Output   |
+---------------------+
```

## 3. Components

| Component | Purpose |
|---|---|
| Frontend | Collects 20 environmental inputs from the user. |
| Flask Backend | Receives requests and coordinates prediction. |

| | |
|---|---|
| Scaler | Normalizes input data before prediction. |
| XGBoost Model | Predicts flood probability. |
| Result Module | Displays probability and Low/Medium/High risk. |

## 4. Data Flow

1. User enters environmental parameters.
2. Flask validates the inputs.
3. Scaler preprocesses the values.
4. XGBoost model predicts flood probability.
5. System displays probability and risk level.

## 5. Conclusion

The layered architecture makes the Rising Water application modular, maintainable, and suitable for future enhancements such as live weather APIs and GIS integration.